Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Летучка № 2

по курсу «Языки и методы программирования»

«Создание "обертки"для веб-сервера»

Студент группы ИУ9-21Б Воронков А. С.

Преподаватель Посевин Д. П.

Москва 2026

1 Цель

Преобразовать "обертку"для веб-сервера согласно индивидиальному варианту

2 Индивидуальный вариант

Реализовать "обертку"веб-сервера для следующих классов:

1. Класс HTTP-кодов и сообщений об ошибках
2. Класс n-мерных векторов

3 Практическая реализация

Код файла Test1.java

```
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.util.HashMap;
import java.util.Map;

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

class HttpStatusCode {
    private final String reason;
    private final int code;

    HttpStatusCode(String reason, int code) {
        this.reason = reason;
        this.code = code;
    }

    String getReason() {
        return reason;
    }

    int getCode() {
        return code;
    }
}

class QueryParser {

    static Map<String, String> parse(String query) {
        Map<String, String> params = new HashMap<>();
        if (query == null || query.isEmpty())
            return params;

        for (String pair : query.split("&")) {
            String[] parts = pair.split("=", 2);
            if (parts.length != 2)
                continue;

            String key = URLDecoder.decode(parts[0], StandardCharsets.UTF_8);
```

```
            String value = URLDecoder.decode(parts[1], StandardCharsets.UTF_8
                );
            params.put(key, value);
        }
        return params;
    }
}

class HttpStatusCodeMapper {

    static HttpStatusCode fromParams(Map<String, String> params) {
        String reason = params.getOrDefault("reason", "Ok");
        int code = Integer.parseInt(params.getOrDefault("code", "200"));
        return new HttpStatusCode(reason, code);
    }
}

class PersonJsonSerializer {

    static String toJson(HttpStatusCode p) {
        int id = System.identityHashCode(p);

        return """
        {
          "code": "%d",
          "reason": %s,
          "objectId": "%s"
        }
        """.formatted(
                p.getCode(),
                escape(p.getReason()),
                Integer.toHexString(id));
    }

    private static String escape(String s) {
        return s.replace("\"", "\\\"");
    }
}

public class Test1 {

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(5678), 0)
            ;
        server.createContext("/test", new MyHandler());
        server.setExecutor(null); // creates a default executor
```

```java
        server.start();
    }

    static class MyHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            if (!"GET".equals(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(405, -1);
                return;
            }

            String query = exchange.getRequestURI().getQuery();
            Map<String, String> params = QueryParser.parse(query);

            HttpStatusCode person;
            try {
                person = HttpStatusCodeMapper.fromParams(params);
            } catch (NumberFormatException e) {
                exchange.sendResponseHeaders(400, -1);
                return;
            }

            String json = PersonJsonSerializer.toJson(person);

            exchange.getResponseHeaders().add("Content-Type", "application/
                json");
            exchange.sendResponseHeaders(200, json.getBytes().length);

            try (OutputStream os = exchange.getResponseBody()) {
                os.write(json.getBytes());
            }
        }
    }
}
```

Код файла Test2.java

```java
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.util.HashMap;
import java.util.Map;

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
```

```java
class Vector {
    private final String vector;

    Vector(String vector) {
        this.vector = vector;
    }

    String getVector() {
        return vector;
    }
}

class QueryParser {

    static Map<String, String> parse(String query) {
        Map<String, String> params = new HashMap<>();
        if (query == null || query.isEmpty())
            return params;

        for (String pair : query.split("&")) {
            String[] parts = pair.split("=", 2);
            if (parts.length != 2)
                continue;

            String key = URLDecoder.decode(parts[0], StandardCharsets.UTF_8);
            String value = URLDecoder.decode(parts[1], StandardCharsets.UTF_8
                );
            params.merge(key, value, (oldVal, newVal) -> oldVal + ", " +
                newVal);
        }
        return params;
    }
}

class VectorMapper {

    static Vector fromParams(Map<String, String> params) {
        String reason = params.getOrDefault("vector", "0");
        return new Vector(reason);
    }
}

class VectorJsonSerializer {

    static String toJson(Vector p) {
```

```java
        int id = System.identityHashCode(p);

        return """
        {
          "vector": "(%s)",
          "objectId": "%s"
        }
        """.formatted(
                p.getVector(),
                Integer.toHexString(id));
    }

    private static String escape(String s) {
        return s.replace("\"", "\\\"");
    }
}

public class Test2 {

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(5678), 0)
            ;
        server.createContext("/test", new MyHandler());
        server.setExecutor(null); // creates a default executor
        server.start();
    }

    static class MyHandler implements HttpHandler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            if (!"GET".equals(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(405, -1);
                return;
            }

            String query = exchange.getRequestURI().getQuery();
            Map<String, String> params = QueryParser.parse(query);

            Vector vector;
            try {vector = VectorMapper.fromParams(params);
            } catch (NumberFormatException e) {
                exchange.sendResponseHeaders(400, -1);
                return;
            }

            String json = VectorJsonSerializer.toJson(vector);
```

```
            exchange.getResponseHeaders().add("Content-Type", "application/
                json");
            exchange.sendResponseHeaders(200, json.getBytes().length);

            try (OutputStream os = exchange.getResponseBody()) {
                os.write(json.getBytes());
            }
        }
    }
}
```

4 Результаты

В результате работы программы получилось запустить "обертки"на веб-серверах для двух классов. Класс HTTP кодов выводит код ошибки, причину и айди объекта. Класс векторов выводит строку координат вектора и айди объекта.

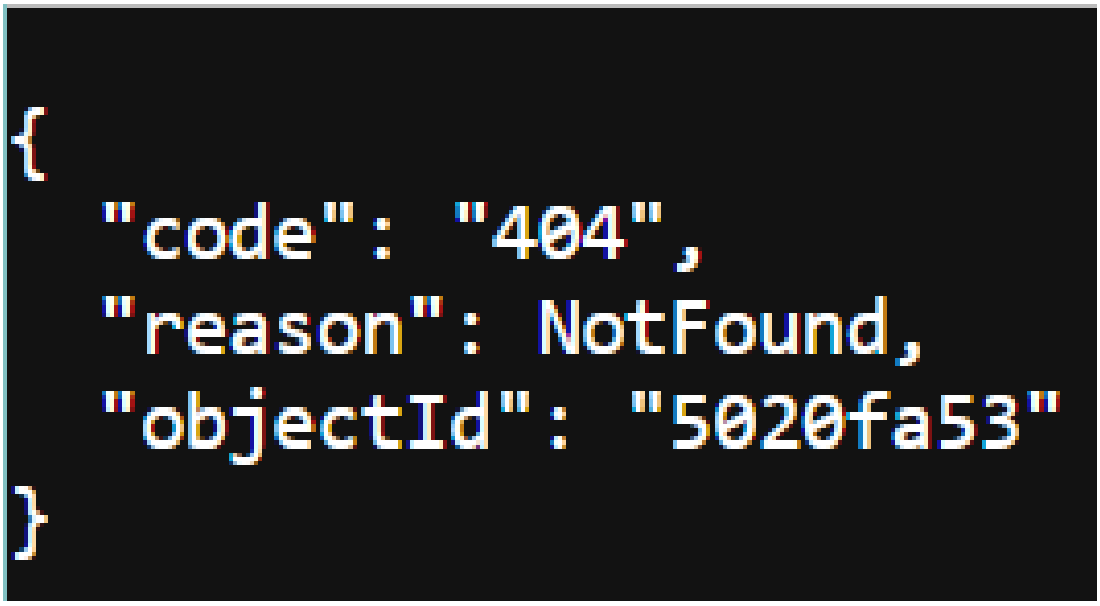

Рис. 1 — Результат запроса с параметрами ошибки:
http://185.102.139.168:5678/test?code=404&reason=NotFound

```
{
  "code": "200",
  "reason": Ok,
  "objectId": "1b0ad141"
}
```


Рис. 2 — Результат запроса без параметров ошибки:
http://185.102.139.168:5678/test


```
{
  "vector": "(1, 2)",
  "objectId": "75ecedf7"
}
```


Рис. 3 — Результат запроса с параметрами вектора:
http://185.102.139.168:5678/test?vector=1&vector=2


```
{
  "vector": "(0)",
  "objectId": "594ff8dd"
}
```

Рис. 4 — Результат запроса без параметров вектора:
http://185.102.139.168:5678/test

5 Вывод

В данной лабораторной работе была изменена "обертка"и переработана под новые классы. Изначальный код был разобран и преобразован под требуемое задание.